S&DS 365 / 665

Intermediate Machine Learning

Variational Autoencoders (VAEs)

October 14

Yale

Version of 2026/10/13 at 21:32

Reminders

- Assignment 2 due tomorrow (midnight)
- Assignment 3 posted tonight
- Midterm Monday in class
- Review sessions: Thurs, Fri

For Today

- Variational inference: The ELBO (rehash)
- Variational autoencoders
- Demo notebook
- Questions for review (AMA)

Variational inference: Strategy

- We'd like to compute $p(\theta, z \mid x)$, but it's too complicated.
- Strategy: Approximate as $q(\theta, z)$ that has a "nice" form
- q is a function of variational parameters, optimized for each x.
- Maximize a lower bound on $p(x)$.

Variational inference: The ELBO

The ELBO is the following lower bound on $\log p(x)$:

$$\log\ p(x) \geq H(q) + \mathbb{E}_q\big(\log p(x, z, \theta)\big)$$

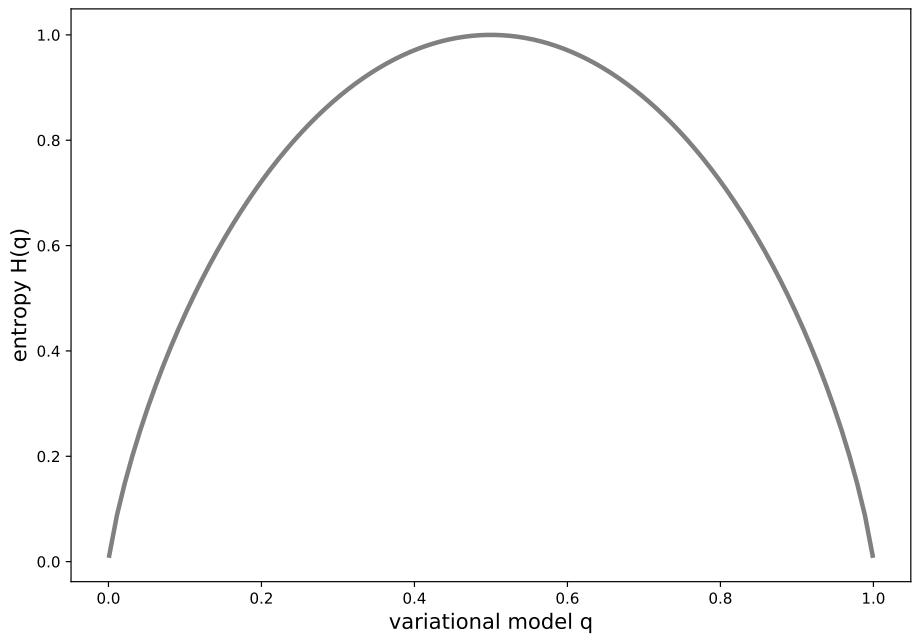
We maximize this over the parameters of q



Variational inference: The ELBO

The ELBO is $H(q) + \mathbb{E}_q(\log p)$

The entropy term $H(q)$ encourages q to be spread out:



The cross-entropy $-\mathbb{E}_q \log p$ tries to match q to p

Variational inference: The ELBO

Since $\log p(x, z, \theta) = \log p(z, \theta) + \log p(x \mid z, \theta)$, the ELBO can also be written as

$$\text{ELBO} = \mathbb{E}_q(\log p(x \mid Z, \theta)) - D_{KL}(q(Z, \theta) \parallel p(Z, \theta))$$

- The Kullback-Leibler divergence term acts as a regularizer, encouraging the variational distribution to be close to the prior

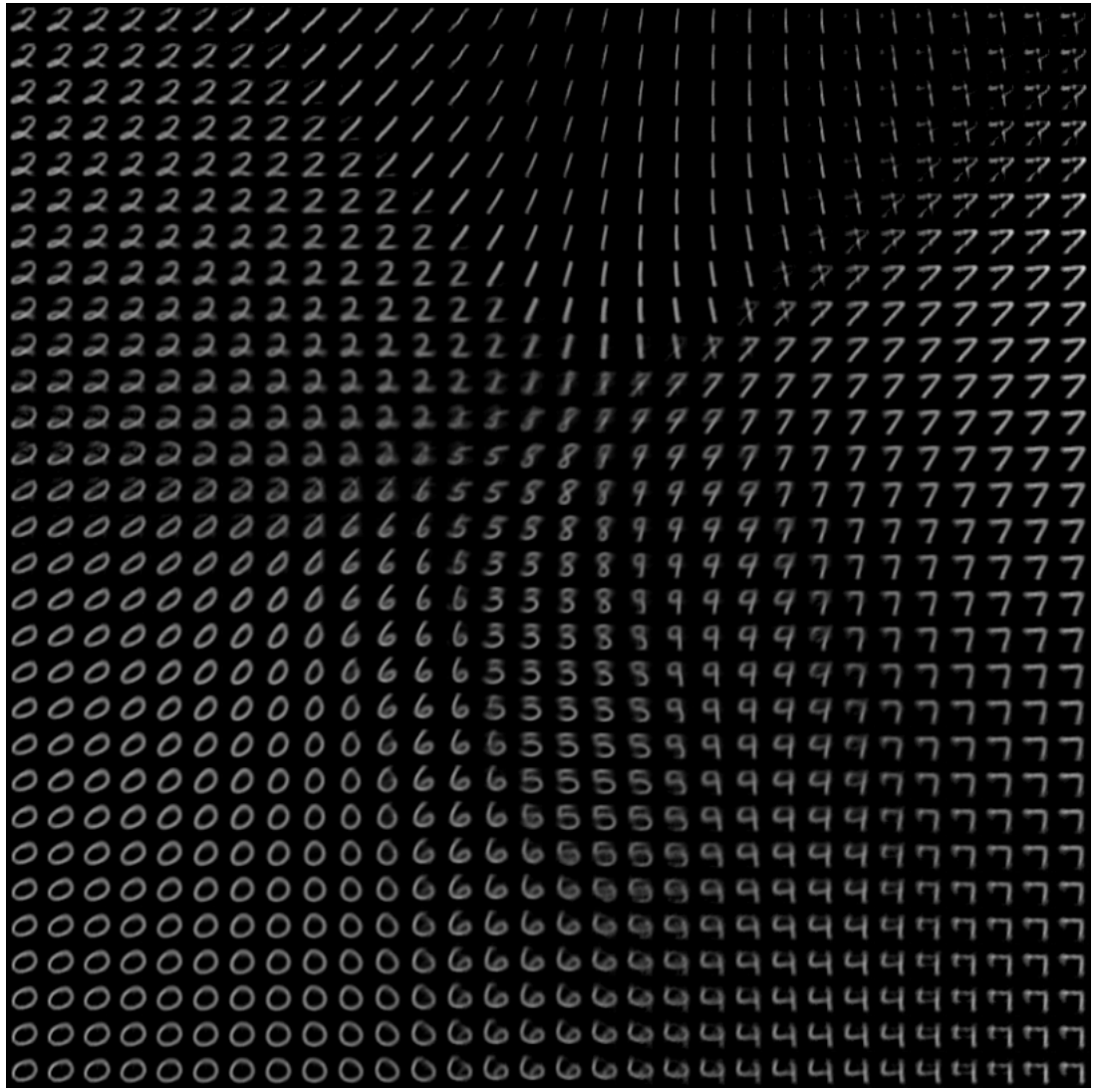
Variational inference: The ELBO

- Note: Separate approximation for each x
- In VAE's we *combine* these estimates together by building them into a neural network, taking x as input.

Variational autoencoders

- A variational autoencoder is an approach to training generative models using variational inference
- The “decoder” is a neural net that generates from a latent variable
  - This is the generative model we want to work with
- The “encoder” approximates the posterior distribution with another neural network trained using variational inference

Visualizing a 2-dim latent space



Variational autoencoders

Start with a generative model

$$z \sim N(0, I_k)$$
$$x \mid z \sim N(G(z), I_d)$$

$G(z)$ is the *generator network* or *decoder*. The latent z is k-dimensional and the output x is d-dimensional.

For example, use a 2-layer network

$$G(z) = A_2\varphi(A_1 z + b_1) + b_2$$

Posterior inference

$$Z \sim N(0, I_k)$$
$$X \mid z \sim N(G(z), I_d)$$

Posterior inference to compute $p(z \mid x)$ is intractable because $G(z)$ is nonlinear (no conjugacy!)

Training the model

How do we train the generative network?

$$Z \sim N(0, I_k)$$
$$X \mid z \sim N(G(z), I_d)$$

Training the model

How do we train the generative network?

$$Z \sim N(0, I_k)$$
$$X \mid z \sim N(G(z), I_d)$$

Make the likelihood of the data as large as possible: minimize the log-loss:

$$-\log p(x) = -\log \int p(z)\, p(x \mid z) dz$$

Again intractable, because $G(\cdot)$ is nonlinear

Training the model

How do we train the generative network?

$$Z \sim N(0, I_k)$$
$$X \mid z \sim N(G(z), I_d)$$

Make the likelihood of the data as large as possible: minimize the log-loss:

$$-\log p(x) = -\log \int p(z)\, p(x \mid z) dz$$

Again intractable, because $G(\cdot)$ is nonlinear

Approach: Use variational inference and the ELBO

Using variational inference

For variational inference we take

$$q(z \mid x) = N(\mu_x, \sigma_x^2 I_k)$$

where now μ_x and σ_x^2 are the *variational parameters*

Using neural networks

Rather than estimate μ_x for each x, we build an encoder neural network that outputs the mean and variance.

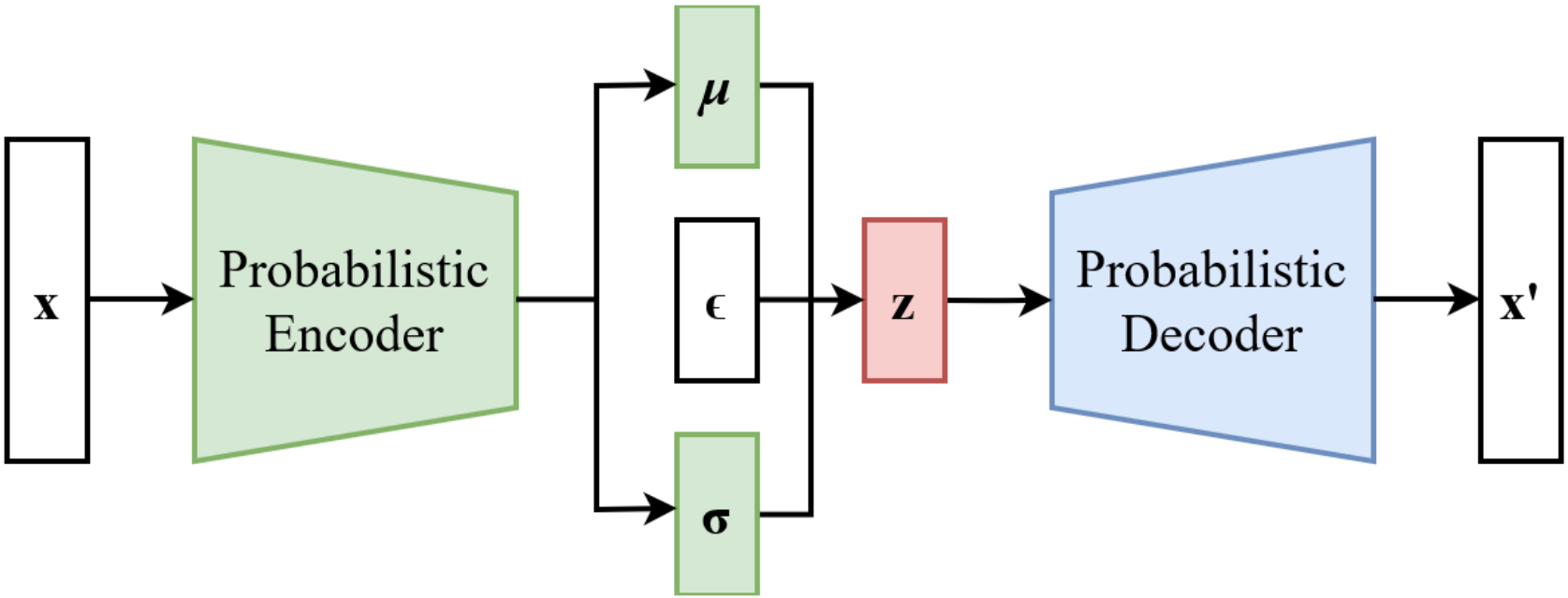
For example:

$$\mu(x) = B_2\,\varphi(B_1 x + d_1) + d_2$$

and similarly for $\log \sigma^2(x)$.

Here φ is an activation function (for example ReLU)

VAE architecture



Training the neural networks

Decoder network: $z \mapsto x$, weights A, b

Encoder network: $x \mapsto \mu(x),\ \log \sigma^2(x)$, weights B, d

Train both networks simultaneously, to maximize the ELBO

- Decoder trained with loss $-\mathbb{E}_q \log p(x, Z_s)$ over A, b
- Encoder trained with (minus) ELBO $-H(q) - \mathbb{E}_q \log p(x, Z_s)$ over B, d

---

The entropy of a multivariate Gaussian with covariance Σ is $\frac{1}{2} \log |\Sigma| + \text{constant}$

The entropy term here is $\log \sigma^2(x) + \text{constant}$, which is taken to be an output of the encoder network.

Using neural networks

Now, approximate $\mathbb{E}_q(\log p(x, Z))$ by sampling (weak law of large numbers)

$$\mathbb{E}_q(\log p(x, Z)) \approx \frac{1}{N} \sum_{s=1}^{N} \log p(x, Z_s)$$

This is fine for the decoder network, but not for training the encoder network using ELBO.

Problem: The parameters of the encoder network have disappeared!

Solution: Reparameterize the samples by $Z_i = \mu(x) + \sigma(x)\epsilon_i$ where $\epsilon_i \sim N(0, I_k)$.

This is called "the reparameterization trick." D. Kingma and M. Welling, "Autoencoding Variational Bayes," https://arxiv.org/abs/1312.6114. The resulting algorithm is an instance of "the EM algorithm."

Simple example (similar to Assn3)

For simplicity, let's take 1-layer networks:

Suppose $x \mid z \sim N(G(z), I)$ where generator network is

$$G(z) = \varphi(Az + b).$$

Then $-\log p(x \mid z)$ is

$$\frac{1}{2}\|x - \varphi(Az + b)\|^2 + \text{constant}$$

Simple example (similar to Assn3)

For simplicity, let's take 1-layer networks:

Suppose $x \mid z \sim N(G(z), I)$ where generator network is

$$G(z) = \varphi(Az + b).$$

Then $-\log p(x \mid z)$ is

$$\frac{1}{2}\|x - \varphi(Az + b)\|^2 + \text{constant}$$

And $-\log p(z)$ is

$$\frac{1}{2}\|z\|^2 + \text{constant}$$

Simple example (continued)

Suppose the approximate posterior is

$$q(z \mid x) = N(\mu(x), \sigma^2(x) I_k)$$

where encoder network is $\mu(x) = \varphi(Bx + d)$. Then

$$-\mathbb{E}_q \log p(x \mid Z) \approx \frac{1}{N} \sum_{s=1}^{N} \frac{1}{2} \|x - \varphi(AZ_s + b)\|^2$$

$$\stackrel{d}{=} \frac{1}{N} \sum_{s=1}^{N} \frac{1}{2} \|x - \varphi(A(\varphi(Bx + d) + \sigma(x)\epsilon_s) + b)\|^2$$

where $\epsilon_s \sim N(0, I_k)$.

Here $\stackrel{d}{=}$ means equal in distribution.

Simple example (continued)

Suppose the approximate posterior is

$$q(z \mid x) = N(\mu(x), \sigma^2(x) I_k)$$

where encoder network is $\mu(x) = \varphi(Bx + d)$. Then

$$\begin{aligned} -\mathbb{E}_q \log p(Z) &= \frac{1}{2}\mathbb{E}_q \|Z\|^2 \\ &= \frac{1}{2}\|\mu(x)\|^2 + \frac{k}{2}\sigma^2(x) \end{aligned}$$

Simple example (continued)

Suppose the approximate posterior is

$$q(z \mid x) = N(\mu(x), \sigma^2(x) I_k)$$

where encoder network is $\mu(x) = \varphi(Bx + d)$. Then

$$H(q) = -\mathbb{E}_q \log q(Z) = \frac{k}{2} \log \sigma^2(x)$$

Resulting training objective

- This gives all of the pieces for the ELBO objective
- You specify this loss function — automatic differentiation (PyTorch) calculates the gradients for you
- The ELBO is optimized by alternating stochastic gradient steps on the weights in the decoder and encoder networks

Demo

Variational Autoencoder on MNIST data

This is a Tensorflow implementation of Variational Autoencoder (VAE) on MNIST data, based on *Auto-Encoding Variational Bayes* (Kingma and Welling 2014).

It uses probabilistic encoders and decoders realized by Multilayer Perceptrons (MLP) with a single hidden layer. The VAE was trained incrementally with mini-batches using partial fit.

The MNIST dataset, distributed by Yann Lecun's THE MNIST DATABASE of handwritten digits website, consists of pair "handwritten digit image" and "label". The image is a gray scale image with 28 x 28 pixels. Pixel values range from 0 (black) to 255 (white), scaled in the [0, 1] interval. The label is the actual digit, ranging from 0 to 9, the image represents.

The original notebook was composed by Jan Hendrik Metzen. Modifications were made by Sunnie Kim on May 24th, 2018.

In [1]:
```
import numpy as np
import tensorflow as tf

import matplotlib.pyplot as plt
%matplotlib inline

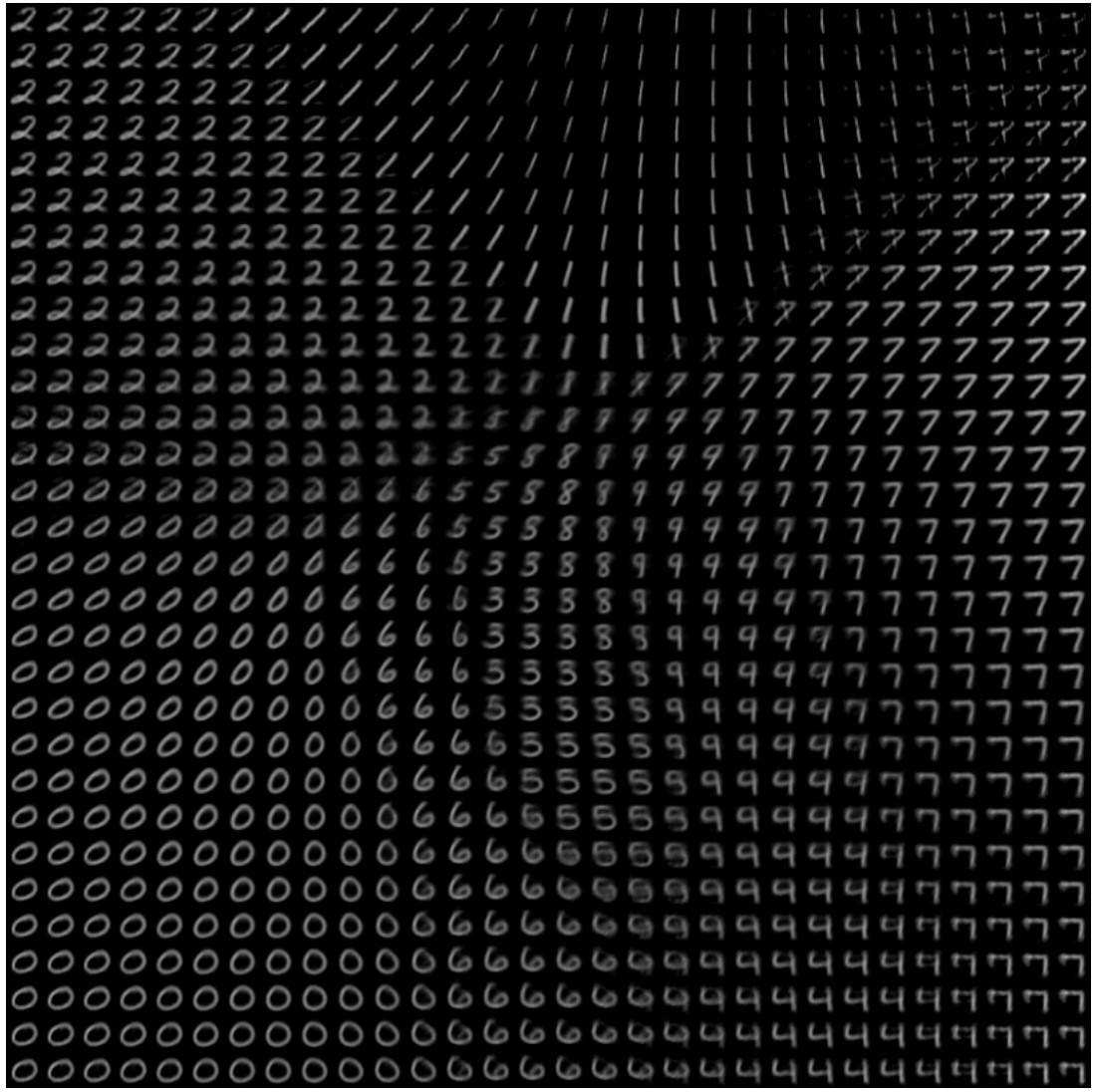
np.random.seed(0)
tf.set_random_seed(0)
```

Load MNIST data in a format suited for Tensorflow

The 'input_data' script is available at: https://raw.githubusercontent.com/tensorflow/tensorflow/master/tensorflow/examples/tutorials/mnist/input_data.py

In [2]:
```
from tensorflow.examples.tutorials.mnist import input_data
tf.logging.set_verbosity(tf.logging.ERROR)
mnist = input_data.read_data_sets("MNIST_data/", one_hot=True)
n_samples = mnist.train.num_examples
```

Visualizing a 2-dim latent space



Deconvolutional neural networks

- A convolutional network goes from a high dimensional input to a lower dimensional output
- The decoder / generator network goes from a low dimensional latent vector to a high dimensional output
- “Deconvolutional” or transposed convolutional networks are the “opposite” of CNNs
- Useful for this type of problem when data are images
- We won’t go into details here

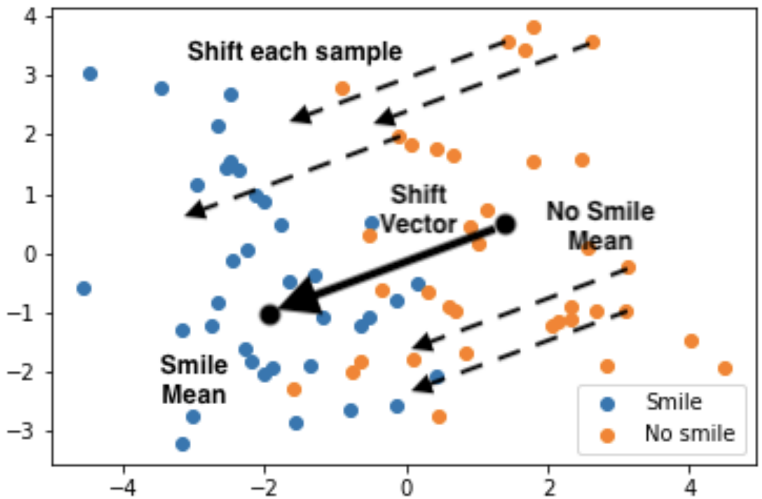
Many tutorials can be found, for example https://towardsdatascience.com/transposed-convolution-demystified-84ca81b4baba

VAEs on Assn3



VAEs on Assn3

Here is a diagram demonstrating the shift in the latent space. Please note that the two-dimensional latent space is just for demonstration purpose. You should *not* use PCA in this problem. Instead, use the original latent space.



Perform attribute shift on at three attributes including smile. Comment on the faces with shifted attributes. (5 points)

- Do the generated faces look like what you expected? If not, can you think of some possible reasons?
- Do the faces with new attributes resemble the original faces? If not, can you think of some possible reasons?
- Which of the attribute shift is more successful? What are some possible reasons?
- Comment on any other aspects of your findings that are interesting to you.

Summary

- Variational methods make deterministic approximations
- General recipe: Maximize ELBO over variational parameters
- VAEs: Generator / decoder is a neural network
- Variational mean is output of a second neural network
- The two networks are trained together to maximize the ELBO